

# Dự Án: Thiết kế CPU MIPS bằng Verilog

## I. GIỚI THIỆU

### 1. MIPS là gì ?

**Kiến trúc MIPS (Microprocessor without Interlocked Pipelined Stages)** là một kiến trúc vi xử lý được thiết kế với đặc điểm không sử dụng cơ chế khóa liên động (interlock) giữa các giai đoạn trong pipeline. Trong các bộ xử lý có pipeline, khi xảy ra xung đột dữ liệu (data hazard), thông thường phần cứng sẽ tự động tạm dừng (stall) để đảm bảo dữ liệu được xử lý chính xác. Tuy nhiên, trong kiến trúc MIPS, cơ chế này không được tích hợp nhằm đơn giản hóa thiết kế phần cứng và tăng hiệu suất xử lý.

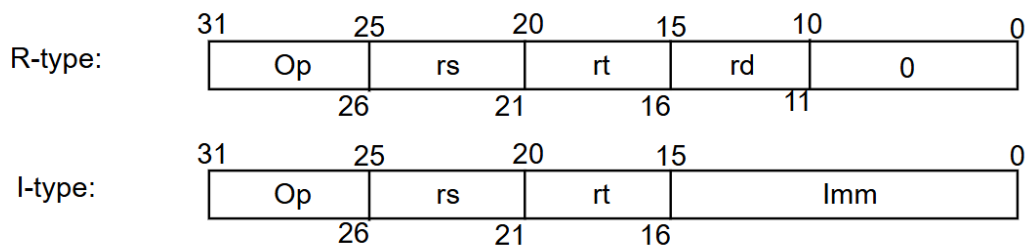
Thay vì dựa vào phần cứng để xử lý các xung đột, MIPS yêu cầu trình biên dịch hoặc lập trình viên chủ động sắp xếp lại thứ tự lệnh hoặc chèn các lệnh rỗng (NOP) để tránh lỗi do phụ thuộc dữ liệu. Cách tiếp cận này giúp giảm độ phức tạp của bộ xử lý, nhưng đồng thời đặt ra yêu cầu cao hơn đối với quá trình tối ưu mã lệnh.

### 2. Tập lệnh ( Instruction Set) sử dụng trong CPU MIPS

Trong thiết kế CPU MIPS bằng Verilog, hệ thống chỉ hỗ trợ một tập con các lệnh cơ bản nhằm đơn giản hóa việc hiện thực phần cứng nhưng vẫn đảm bảo minh họa đầy đủ nguyên lý hoạt động của kiến trúc MIPS. Các lệnh được chia thành hai định dạng chính là **R-type** và **I-type**, tương ứng với cách mã hóa và chức năng xử lý khác nhau.

#### 1. Phân loại định dạng lệnh

- **R-type (Register type):**  
Các phép toán được thực hiện hoàn toàn trên thanh ghi. Kết quả được ghi vào thanh ghi đích (rd).
- **I-type (Immediate type):**  
Sử dụng thêm một giá trị tức thời (immediate) trong quá trình tính toán hoặc truy cập bộ nhớ.



Hình 1. Trường dữ liệu của tập lệnh

#### 2. Các lệnh được hỗ trợ

- **add (R-type):**  
Thực hiện phép cộng hai thanh ghi. **Ký hiệu: add \$rd, \$rs, \$rt**

$$R[rd] = R[rs] + R[rt]$$

- **addi (I-type):**  
Cộng giá trị thanh ghi với một hằng số (immediate). **Ký hiệu: addi \$rt, \$rs, imm**

$$R[rt] = R[rs] + \text{SignExtImm}$$

- **slt (R-type):**

So sánh hai thanh ghi, nếu nhỏ hơn thì gán 1, ngược lại gán 0. **Ký hiệu: slt \$rd, \$rs, \$rt**

$$R[rd] = (R[rs] < R[rt])? 1: 0$$

- **lw (Load Word – I-type):**

Đọc dữ liệu từ bộ nhớ và ghi vào thanh ghi. **Ký hiệu: lw \$rt, imm(\$rs)**

$$R[rt] = M[R[rs] + SignExtImm]$$

- **sw (Store Word – I-type):**

Ghi dữ liệu từ thanh ghi vào bộ nhớ. **Ký hiệu: sw \$rt, imm(\$rs)**

$$M[R[rs] + SignExtImm] = R[rt]$$

### 3. Ý nghĩa các trường trong instruction

Do thiết kế có số lượng lệnh được hỗ trợ nhỏ ( chỉ gồm 5 lệnh). Do đó, Một instruction 32-bit trong MIPS sẽ khác so với thiết của ISA:

- **opcode (31–26):** xác định lệnh
- **rs (25–21):** thanh ghi nguồn 1
- **rt (20–16):** thanh ghi nguồn 2 hoặc đích (với I-type)
- **rd (15–11):** thanh ghi đích (R-type)
- **shamt (10–6):** dịch bit (không dùng trong các lệnh hiện tại)
- **funct (5–0):** xác định phép toán cụ thể (không dùng trong các lệnh hiện tại)
- **immediate (15–0):** giá trị hằng (I-type)

### 4. Các khối hỗ trợ thực thi instruction

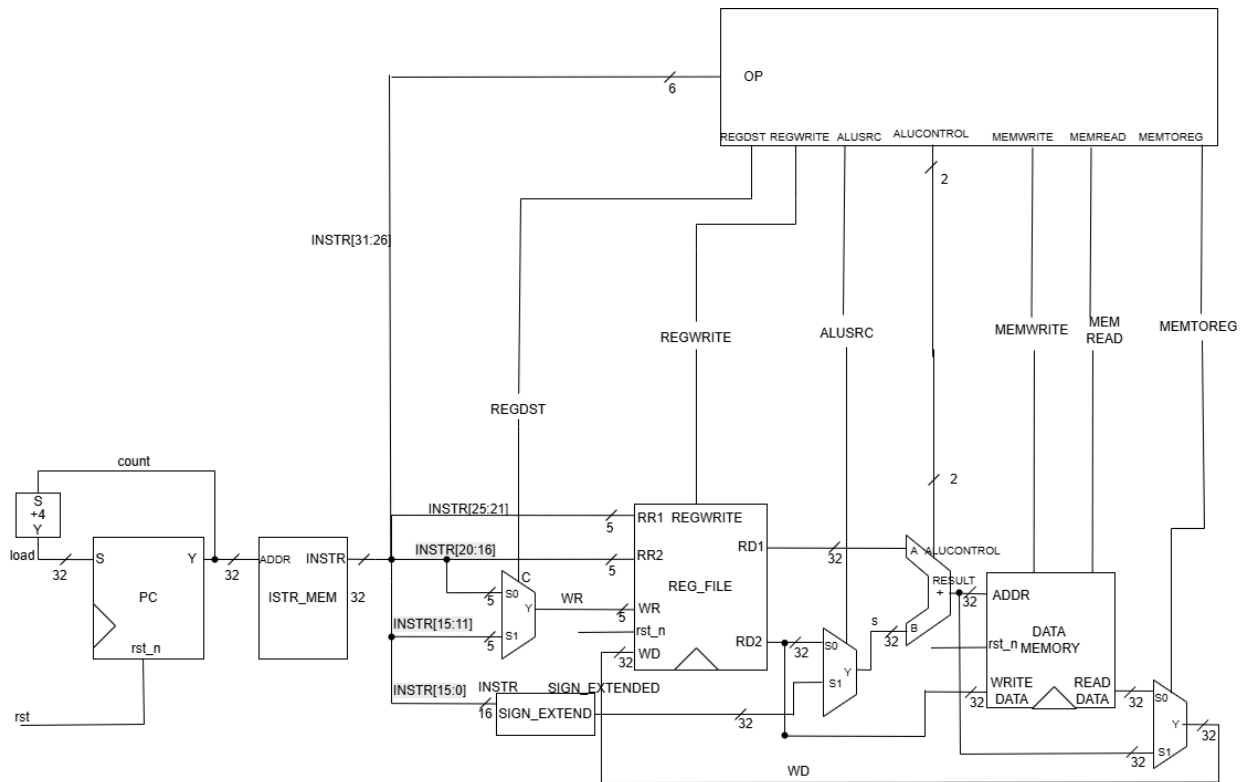
Để thực hiện các lệnh trên, CPU sử dụng các khối chức năng chính:

- **ALU (Arithmetic Logic Unit):** thực hiện các phép toán số học và logic (add, slt)
- **Register File:** lưu trữ các thanh ghi và cung cấp dữ liệu đầu vào/đầu ra
- **Data Memory:** dùng cho các lệnh lw và sw
- **Sign Extension:** mở rộng dấu cho giá trị immediate từ 16-bit lên 32-bit
- **MUX (Multiplexer):** chọn dữ liệu đầu vào phù hợp (ví dụ: chọn rd hoặc rt)
- **Control Unit:** tạo tín hiệu điều khiển dựa trên opcode

## II. CPU

### 1. Sơ đồ khối

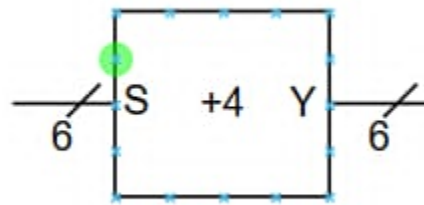
Ở trên hình, nhóm chúng em đã rút gọn tín hiệu clk bằng dấu tam giác.



Hình 2. Sơ đồ khối của thiết kế

## 2. Các khối con

### 2.1. Bộ cộng 4

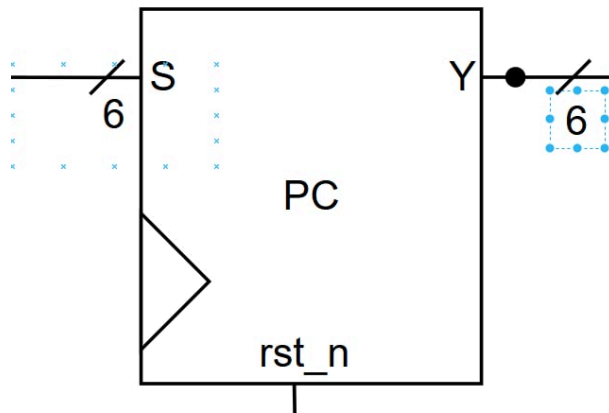


Hình 3. Sơ đồ khối của bộ cộng 4

| Tên | Độ dài (bit) | Loại    | Chức năng        |
|-----|--------------|---------|------------------|
| S   | 6            | Đầu vào | Trước khi cộng 4 |
| Y   | 6            | Đầu ra  | Sau khi cộng 4   |

Chức năng: Cộng 4 vào giá trị hiện tại

## 2.2. PC

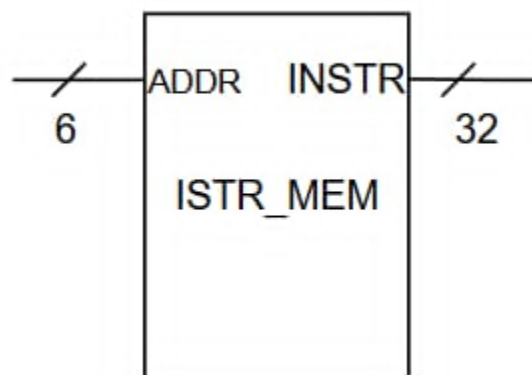


Hình 4. Sơ đồ khối của Program Counter

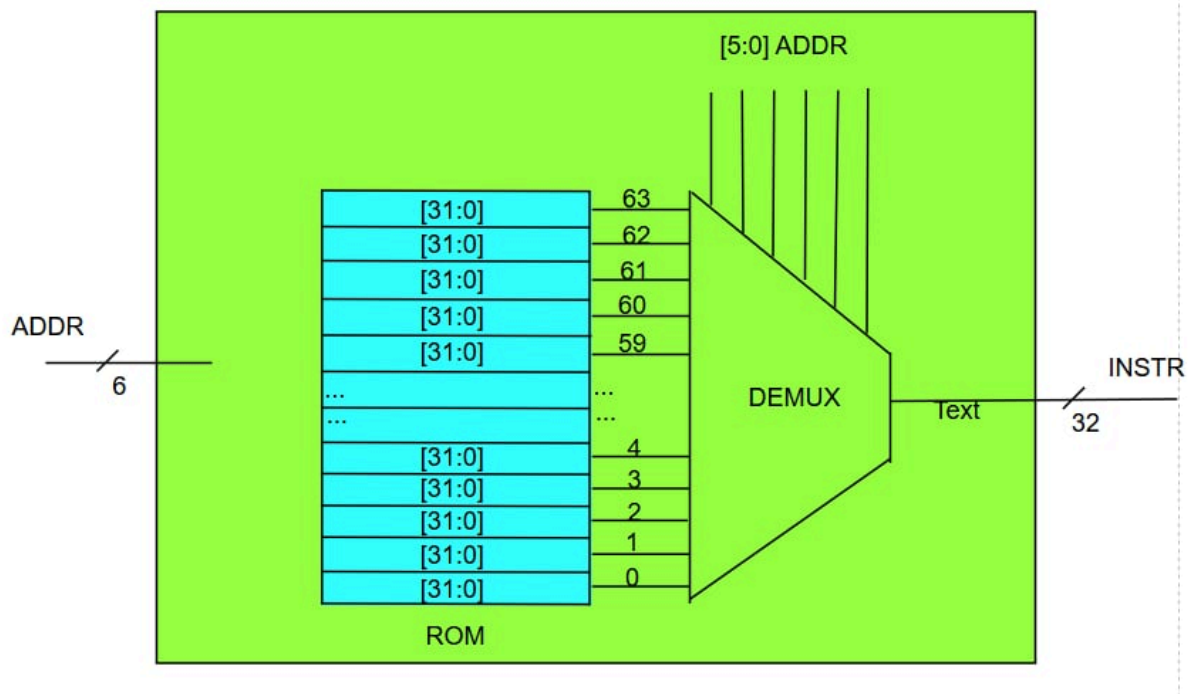
| Tên   | Độ dài (bit) | Loại    | Chức năng                                     |
|-------|--------------|---------|-----------------------------------------------|
| S     | 6            | Đầu vào | Vị trí cho tập lệnh kế tiếp                   |
| Y     | 6            | Đầu ra  | Vị trí cho tập lệnh hiện tại                  |
| rst_n | 1            | Đầu vào | reset vị trí về 0                             |
| clk   | 1            | Đầu vào | cung cấp tín hiệu clock nhằm đồng bộ hệ thống |

Chức năng: Đóng vai trò là bộ đếm cho chương trình, nhằm biết lệnh tiếp theo cần thực hiện.

## 2.3. ISTR\_MEM



Hình 5. Sơ đồ khối của ISTR\_MEM

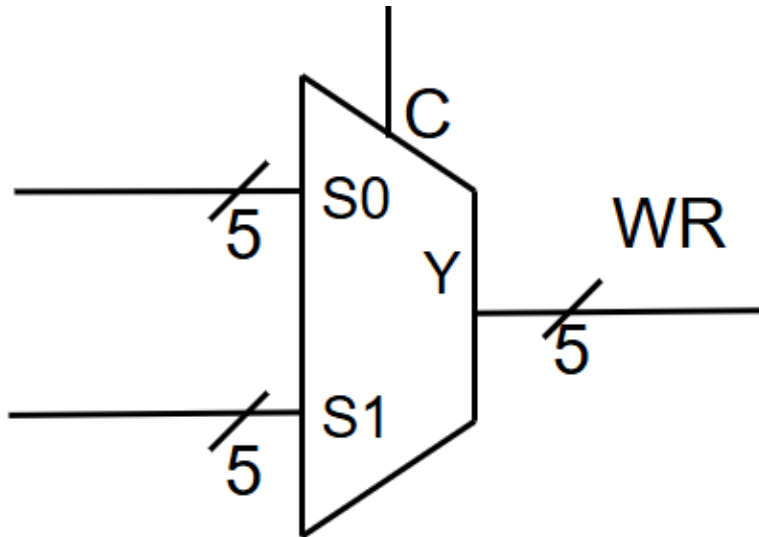


Hình 6. Sơ đồ khối chi tiết của ISTR\_MEM

| Tên   | Độ dài (bit) | Loại    | Chức năng                      |
|-------|--------------|---------|--------------------------------|
| ADDR  | 6            | Đầu vào | địa chỉ của lệnh cần thực hiện |
| INSTR | 32           | Đầu ra  | Lệnh cần thực hiện             |

Chức năng: Chứa mã nguồn ( nhiều tập lệnh) của chương trình. Được nạp sẵn ban đầu.

#### 2.4. MUX 2-1 5 bit



Hình 7. Sơ đồ khối của MUX 2-1 5 bit

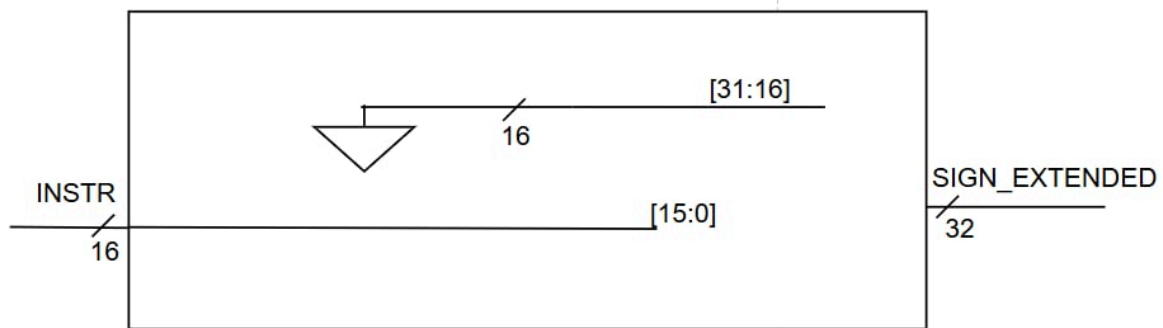
| Tên | Độ dài (bit) | Loại    | Chức năng          |
|-----|--------------|---------|--------------------|
| S0  | 5            | Đầu vào | Được chọn nếu C =0 |
| S1  | 5            | Đầu vào | Được chọn nếu C =1 |
| C   | 1            | Đầu vào | Tín hiệu chọn      |
| Y   | 5            | Đầu ra  | Tín hiệu đầu ra    |

Chức năng: Chọn 1 trong 2 tín hiệu đầu vào, phụ thuộc tín hiệu đầu ra.

## 2.5. SIGN\_EXTEND



Hình 8. Sơ đồ khối của SIGN\_EXTEND

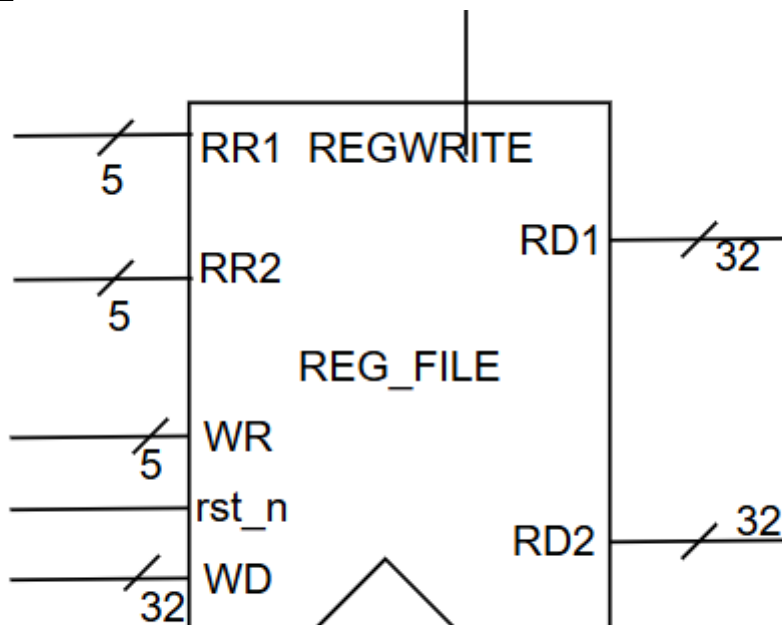


Hình 9. Sơ đồ khối chi tiết của SIGN\_EXTEND

| Tên           | Độ dài (bit) | Loại    | Chức năng                    |
|---------------|--------------|---------|------------------------------|
| INSTR         | 16           | Đầu vào | giá trị Immediately          |
| SIGN_EXTENDED | 32           | Đầu ra  | Giá trị sau khi được mở rộng |

Chức năng: Mở rộng bit dấu của giá trị đầu vào, từ 16 bit thành 32 bit để các thao tác phía sau không bị lỗi.

## 2.6. REG\_FILE



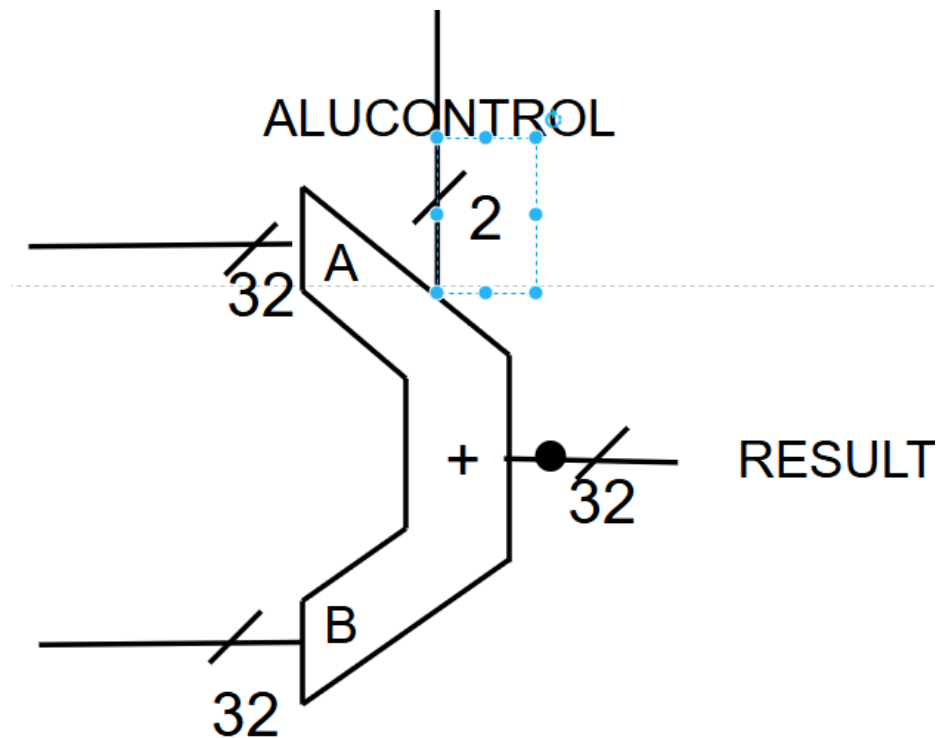
Hình 10. Sơ đồ khối của REG\_FILE

| Tên | Độ dài (bit) | Loại | Chức năng |
|-----|--------------|------|-----------|
|-----|--------------|------|-----------|

|          |    |         |                                               |
|----------|----|---------|-----------------------------------------------|
| RR1      | 5  | Đầu vào | Địa chỉ chọn thanh ghi để đọc dữ liệu số nhất |
| RR2      | 5  | Đầu vào | Địa chỉ chọn thanh ghi để đọc dữ liệu số hai  |
| WR       | 5  | Đầu vào | Địa chỉ chọn để ghi vào                       |
| WD       | 32 | Đầu vào | Địa chỉ chọn thanh ghi để ghi vào             |
| rst_n    | 1  | Đầu vào | Tín hiệu reset tích cực thấp                  |
| REGWRITE | 1  | Đầu vào | Tín hiệu cho phép ghi vào thanh ghi           |
| RD1      | 32 | Đầu ra  | Giá trị đọc ra từ thanh ghi thứ nhất          |
| RD2      | 32 | Đầu ra  | Giá trị đọc ra từ thanh ghi thứ hai           |

Chức năng: Giống như cache, hỗ trợ việc tính toán tạm thời. Nhanh hơn so với việc đọc dữ liệu trên RAM

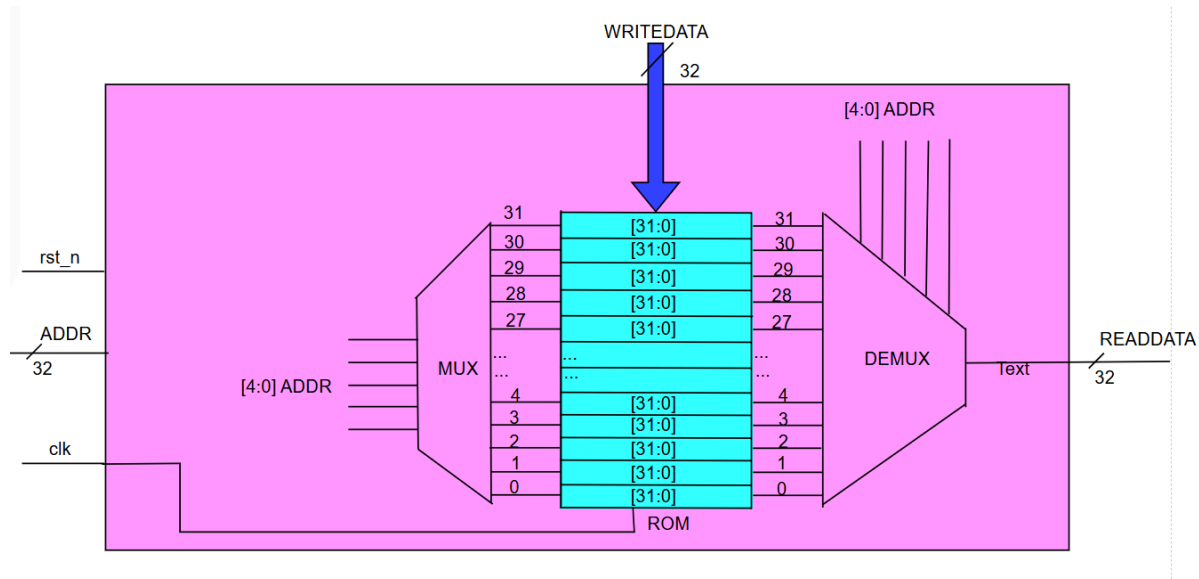
## 2.7. ALU



Hình 11. Sơ đồ khối của ALU





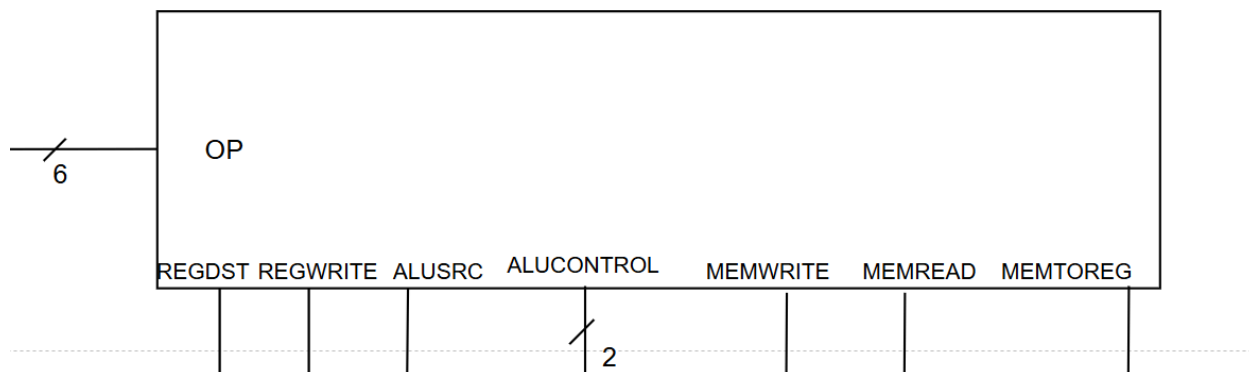


Hình 14. Sơ đồ khối chi tiết của DATA MEMORY

| Tên        | Độ dài (bit) | Loại    | Chức năng                        |
|------------|--------------|---------|----------------------------------|
| ADDR       | 32           | Đầu vào | Địa chỉ để ghi vào hoặc đọc ra   |
| rst_n      | 1            | Đầu vào | Tín hiệu reset tích cực thấp     |
| WRITE DATA | 32           | Đầu vào | Dữ liệu để ghi vào               |
| MEM WRITE  | 1            | Đầu vào | Tín hiệu điều khiển việc ghi vào |
| MEM READ   | 1            | Đầu vào | Tín hiệu điều khiển việc đọc ra  |
| READ DATA  | 32           | Đầu ra  | Dữ liệu đọc ra                   |

Chức năng: Dùng để lưu trữ dữ liệu khi thực hiện chương trình

## 2.9. CONTROL UNIT



Hình 15. Sơ đồ khối của CONTROL UNIT

| Tên        | Độ dài (bit) | Loại    | Chức năng                                        |
|------------|--------------|---------|--------------------------------------------------|
| OP         | 6            | Đầu vào | Thuật toán cần thực thi                          |
| REGDST     | 1            | Đầu ra  | Tín hiệu chọn địa chỉ cho thanh ghi đích         |
| REGWRITE   | 1            | Đầu ra  | Tín hiệu điều khiển việc ghi vào REG_FILE        |
| ALUSRC     | 1            | Đầu ra  | Tín hiệu điều khiển cho MUX chọn dữ liệu vào ALU |
| ALUCONTROL | 2            | Đầu ra  | Tín hiệu điều khiển ALU                          |
| MEMWRITE   | 1            | Đầu ra  | Tín hiệu điều khiển cho khối DATAMEMORY          |
| MEMREAD    | 1            | Đầu ra  | Tín hiệu điều khiển cho khối DATAMEMORY          |
| MEMTOREG   | 1            | Đầu ra  | Tín hiệu điều khiển cho khối REG_FILE            |

| Opcode | Lệnh     | ALUControl | RegDst | MemRead | MemWrite | MemToReg | ALUSrc | RegWrite |
|--------|----------|------------|--------|---------|----------|----------|--------|----------|
| 000001 | add (R)  | 10         | 1      | 0       | 0        | 0        | 0      | 1        |
| 000010 | sw (I)   | 10         | X      | 0       | 1        | X        | 1      | 0        |
| 000100 | lw (I)   | 10         | 0      | 1       | 0        | 1        | 1      | 1        |
| 001000 | addi (I) | 10         | 0      | 0       | 0        | 0        | 1      | 1        |
| 010000 | slt (R)  | 11         | 1      | 0       | 0        | 0        | 0      | 1        |

Chức năng: tạo giá trị điều khiển cho các khối nằm trong DATAPATH.

### III. UART system

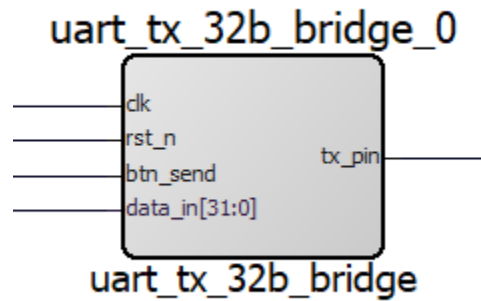
#### 1. UART là gì ?

UART (Universal Asynchronous Receiver Transmitter) là một chuẩn truyền thông nối tiếp (serial communication) không đồng bộ, được sử dụng để trao đổi dữ liệu giữa các thiết bị điện tử như vi điều khiển, máy tính, module GPS, Bluetooth, v.v.

#### 2. Nguyên lý hoạt động

UART hoạt động bằng cách truyền dữ liệu nối tiếp từng bit thông qua một khung dữ liệu gồm start bit, data bits, parity bit và stop bit. Bên nhận phát hiện start bit để đồng bộ và lấy mẫu dữ liệu theo chu kỳ baud nhằm khôi phục lại dữ liệu ban đầu. Do không sử dụng xung clock chung, UART yêu cầu các thiết bị phải cấu hình cùng tốc độ truyền để đảm bảo độ chính xác.

### 3. Cấu trúc



Hình 16. Cấu trúc UART

| Tên      | Độ dài (bit) | Loại    | Chức năng                                  |
|----------|--------------|---------|--------------------------------------------|
| clk      | 1            | Đầu vào | xung clokc                                 |
| rst_n    | 1            | Đầu vào | Tín hiệu reset tích cực thấp               |
| btn_send | 1            | Đầu vào | Tín hiệu thông báo cho việc đẩy dữ liệu ra |
| data_in  | 32           | Đầu vào | Dữ liệu cần đẩy ra màn hình Realterm       |
| tx_pin   | 1            | Đầu ra  | Chân truyền 1 bit ra ngoài                 |

### 4. Code verilog

```

1  module uart_tx_32b_bridge (
2      input wire clk,
3      input wire rst_n,
4      input wire btn_send,
5      input wire [31:0] data_in,
6      output wire tx_pin
7  );
8      reg [19:0] db_cnt;
9      reg btn_dly1, btn_dly2;
10     wire btn_pulse;
11
12     always @(posedge clk or negedge rst_n) begin
13         if (!rst_n) begin
14             db_cnt <= 0;
15             btn_dly1 <= 0;
16             btn_dly2 <= 0;
17         end else begin
18             btn_dly1 <= btn_send;
19             btn_dly2 <= btn_dly1;
20             if (btn_dly1 == 1 && btn_dly2 == 0)

```

Khai báo tín hiệu input là 32 bit data in, output là 1 bit tx

Khởi chống rung nút nhấn ( debounce logic)

```

20         if (btn_dly1 == 1 && btn_dly2 == 0)
21             db_cnt <= 20'd1_000_000;
22         else if (db_cnt > 0)
23             db_cnt <= db_cnt - 1;
24     end
25 end
26
27
28 assign btn_pulse = (db_cnt == 1);
29 reg [12:0] baud_count;
30 reg baud_tick;
31
32 always @(posedge clk or negedge rst_n) begin
33     if (!rst_n) begin
34         baud_count <= 0;
35         baud_tick <= 0;
36     end else if (baud_count == 13'd5207) begin
37         baud_count <= 0;
38         baud_tick <= 1'b1;
39     end else begin
40         baud_count <= baud_count + 1;
41         baud_tick <= 1'b0;
42     end
43 end
44
45 localparam S_IDLE      = 3'd0;
46 localparam S_LOAD      = 3'd1;
47 localparam S_SEND_BYTE = 3'd2;
48 localparam S_WAIT_BUSY = 3'd3;
49 localparam S_WAIT_FREE = 3'd4;
50
51 reg [2:0] state;
52 reg [31:0] data_buf;
53 reg [1:0] byte_cnt;
54 reg [7:0] tx_data_byte;
55 reg tx_start;
56 wire tx_busy;
57
58 always @(posedge clk or negedge rst_n) begin
59     if (!rst_n) begin
60         state <= S_IDLE;
61         tx_start <= 1'b0;
62         byte_cnt <= 0;
63         data_buf <= 0;
64         tx_data_byte <= 0;
65     end else begin
66         tx_start <= 1'b0;
67

```

Khởi khởi tạo tốc độ  
baud với tốc độ bằng  
9600 bit/s

Mã hóa trạng thái

FSM chia nhỏ dữ liệu, để  
chia 32 bit tín hiệu vào  
thành từng đoạn 8 bit để  
có thể truyền ra thông  
qua uart



|    |  |                                        |
|----|--|----------------------------------------|
| 68 |  | case (state)                           |
| 69 |  | S_IDLE: begin                          |
| 70 |  | if (btn_pulse) begin                   |
| 71 |  | data_buf <= data_in;                   |
| 72 |  | byte_cnt <= 2'd3;                      |
| 73 |  | state <= S_LOAD;                       |
| 74 |  | end                                    |
| 75 |  | end                                    |
| 76 |  |                                        |
| 77 |  | S_LOAD: begin                          |
| 78 |  | case (byte_cnt)                        |
| 79 |  | 2'd3: tx_data_byte <= data_buf[31:24]; |
| 80 |  | 2'd2: tx_data_byte <= data_buf[23:16]; |
| 81 |  | 2'd1: tx_data_byte <= data_buf[15:8];  |
| 82 |  | 2'd0: tx_data_byte <= data_buf[7:0];   |
| 83 |  | endcase                                |
| 84 |  | tx_start <= 1'b1;                      |
| 85 |  | state <= S_WAIT_BUSY;                  |
| 86 |  | end                                    |
| 87 |  |                                        |
| 88 |  | S_WAIT_BUSY: begin                     |
| 89 |  | tx_start <= 1'b0;                      |
| 90 |  | if (tx_busy)                           |
| 91 |  | state <= S_WAIT_FREE;                  |
| 92 |  | end                                    |
| 93 |  |                                        |
| 94 |  | S_WAIT_FREE: begin                     |
| 95 |  | if (!tx_busy) begin                    |
| 96 |  | if (byte_cnt == 0) begin               |
| 97 |  | state <= S_IDLE;                       |

```

98     end else begin
99         byte_cnt <= byte_cnt - 1;
100         state <= S_LOAD;
101     end
102 end
103 end
104
105     default: state <= S_IDLE;
106 endcase
107 end
108 end
109
110 uart_tx_core u_tx_core (
111     .clk      (clk),
112     .rst_n    (rst_n),
113     .baud_tick (baud_tick),
114     .tx_start  (tx_start),
115     .tx_data   (tx_data_byte),
116     .tx_pin    (tx_pin),
117     .tx_busy   (tx_busy)
118 );
119

```



```

120 endmodule
121 module uart_tx_core (
122     input clk, rst_n, baud_tick, tx_start,
123     input [7:0] tx_data,
124     output reg tx_pin, tx_busy
125 );
126     reg [3:0] bit_cnt;
127     reg [7:0] shift_reg;
128     reg [1:0] state;
129
130     always @(posedge clk or negedge rst_n) begin
131         if (!rst_n) begin
132             state <= 0; tx_pin <= 1; tx_busy <= 0;
133         end else begin
134             shift_reg <= shift_reg;
135             bit_cnt <= bit_cnt;
136             case (state)
137             0: begin // IDLE
138                 tx_pin <= 1;
139                 if (tx_start) begin
140                     shift_reg <= tx_data;
141                     tx_busy <= 1;
142                     state <= 1;
143                 end else tx_busy <= 0;
144             end
145             1: begin // START BIT
146                 if (baud_tick) begin
147                     tx_pin <= 0;
148                     bit_cnt <= 0;
149                     state <= 2;
150                 end
151             end
152             2: begin // DATA BITS
153                 if (baud_tick) begin
154                     tx_pin <= shift_reg[bit_cnt];
155                     if (bit_cnt == 7) state <= 3;
156                     else bit_cnt <= bit_cnt + 1;
157                 end
158             end
159             3: begin // STOP BIT
160                 if (baud_tick) begin
161                     tx_pin <= 1;
162                     state <= 0;
163                 end
164             end
165         endcase
166     end
167 end
168 endmodule

```

Module uart\_tx\_core là bộ phát UART 8 bit nhận các đoạn 8 bit đã chia để truyền ra ngoài.

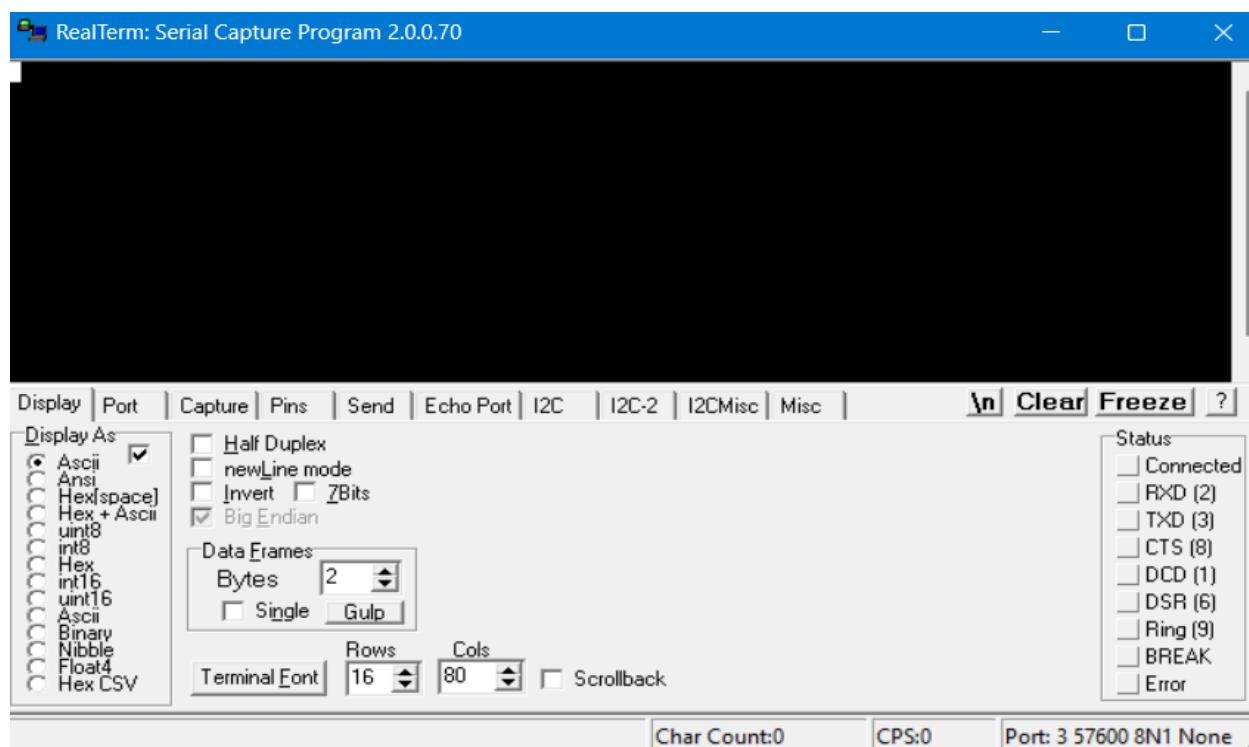
## IV. REALTERM

### 1. Realterm là gì?

RealTerm là một phần mềm **terminal giao tiếp cổng nối tiếp (SERIAL / COM port)** trên Windows, dùng để làm việc với các giao thức như UART, RS-232, USB-to-Serial.

Realterm làm việc tốt với dữ liệu **dạng nhị phân (Binary)**, cho phép debug giao tiếp UART rất chi tiết. Realterm có nhiều cách hiển thị kiểu dữ liệu như ASCII, HEX, Decimal nhưng ở đây với việc đọc dữ liệu 32 bit thì em chọn hiển thị HEX để kiểm tra dữ liệu dễ dàng hơn.

Ở đây, nhóm em sử dụng để có thể đọc dữ liệu ra từ DATA MEMORY thông qua giao thức UART với tốc độ truyền là 9600 bit/s.



Hình 17. Giao diện của phần mềm Realterm

## V. Chương trình Biên dịch

Do tập lệnh được nhóm em rút gọn để phù hợp với mức độ của môn học, nên nhóm em đã tạo ra một trình biên dịch hỗ trợ cho phép tính phương trình:

$$y = a \cdot x + b$$

Nhóm em sử dụng ngôn ngữ C++ để tạo ra các tập lệnh, trong đó:

1. Sử dụng lệnh addi để tải các dữ liệu **[a]**, **[x]**, **[b]** vào các thanh ghi tương ứng là **r2**, **r1**, **r3**.
2. Sử dụng C++ để tạo ra vòng lặp bằng nhiều phép cộng dưới dạng lệnh add, bởi vì kiến trúc này không hỗ trợ phép nhân

3. Sử dụng lệnh sw và lw để hiển thị có thể lưu dữ liệu vào **DATA MEMORY (RAM)** để có thể thông qua UART và truyền ra màn hình.

Code:

```
#include <iostream>
#include <stdint>
#include <iomanip>
#include <string>
```

```
using namespace std;
```

```
// R-type: Opcode(6) | Rs(5) | Rt(5) | Rd(5) | Pad(11)
```

```
uint32_t r_type(int opcode, int rs, int rt, int rd) {
    return (opcode << 26) | (rs << 21) | (rt << 16) | (rd << 11);
}
```

```
// I-type: Opcode(6) | Rs(5) | Rt(5) | Imm(16)
```

```
uint32_t i_type(int opcode, int rs, int rt, int imm) {
    return (opcode << 26) | (rs << 21) | (rt << 16) | (imm & 0xFFFF);
}
```

```
// Hàm hỗ trợ in ra 1 dòng lệnh Verilog chuẩn form
```

```
void printVerilogLine(int addr, uint32_t inst, string comment) {
    cout << "        6'd" << addr << ": q = 32'h"
        << hex << uppercase << setfill('0') << setw(8) << inst << dec
        << "; // " << comment << "\n";
}
```

```
int main() {
    int a, x, b;
```

```

cout << "Nhap a: "; cin >> a;
cout << "Nhap x: "; cin >> x;
cout << "Nhap b: "; cin >> b;

int OP_ADD = 0b000001;
int OP_SW = 0b000010;
int OP_LW = 0b000100;
int OP_ADDI = 0b001000;

cout << "\n===== KET QUA FILE VERILOG =====\n\n";
cout << "module IMEM32 (\n";
cout << "    input wire [5:0] addr, \n";
cout << "    output reg [31:0] q\n";
cout << ");\n";
cout << "    always @(*) begin\n";
cout << "        case (addr[5:0])\n";

int addr = 0;
uint32_t inst;

// --- KHỞI TẠO ---
cout << "        // --- KHOI TAO ---\n";

inst = i_type(OP_ADDI, 0, 4, 0);
printVerilogLine(addr, inst, "addi $4, $0, 0 -> y = 0 ($4 la accumulator)");
addr += 4;

inst = i_type(OP_ADDI, 0, 1, x);

```

```

printVerilogLine(addr, inst, "addi $1, $0, " + to_string(x) + " -> x = " + to_string(x));
addr += 4;

inst = i_type(OP_ADDI, 0, 2, a);
printVerilogLine(addr, inst, "addi $2, $0, " + to_string(a) + " -> a = " + to_string(a) + " (bien
dem)");
addr += 4;

inst = i_type(OP_ADDI, 0, 3, b);
printVerilogLine(addr, inst, "addi $3, $0, " + to_string(b) + " -> b = " + to_string(b));
addr += 4;

// --- LOOP ---
cout << "\n          // --- LOOP ---\n";
int loop_count = 1;
int a_temp = a;
while (a_temp > 0) {
    cout << "          // Lan " << loop_count++ << ": \n";

    inst = r_type(OP_ADD, 4, 1, 4);
    printVerilogLine(addr, inst, "add $4, $4, $1 -> y = y + x");
    addr += 4;

    inst = i_type(OP_ADDI, 2, 2, -1);
    printVerilogLine(addr, inst, "addi $2, $2, -1 -> a = a - 1");
    addr += 4;

    a_temp--;
}

```

```

// --- END ---

cout << "\n          // --- END ---\n";
inst = r_type(OP_ADD, 4, 3, 5);
printVerilogLine(addr, inst, "add $5, $4, $3 -> y = y + b");
addr += 4;

inst = i_type(OP_SW, 0, 5, 0);
printVerilogLine(addr, inst, "sw $5, 0($0) -> Luu ket qua vao Word 0");
addr += 4;

uint32_t trap_inst = i_type(OP_LW, 0, 6, 0);
printVerilogLine(addr, trap_inst, "lw $6, 0($0) -> Doc Word 0 ra $6 (READDATA)");

// --- DEFAULT ---

cout << "\n          // TRAP:\n";

cout << "          default: q = 32'h" << hex << uppercase << setfill('0') << setw(8) << trap_inst
<< dec << "; \n";

// --- FOOTER ---

cout << "          endcase\n";
cout << "        end\n";
cout << "endmodule\n";
cout << "\n===== \n";

return 0;
}

```

**Kết quả thu được:**

```

module IMEM32 (
    input wire [5:0] addr,
    output reg [31:0] q
);
    always @(*) begin
        case (addr[5:0])
            // --- KHOI TAO ---
            6'd0: q = 32'h20040000; // addi $4, $0, 0 -> y = 0 ($4 la accumulator)
            6'd4: q = 32'h20010002; // addi $1, $0, 2 -> x = 2
            6'd8: q = 32'h20020002; // addi $2, $0, 2 -> a = 2 (bien dem)
            6'd12: q = 32'h20030002; // addi $3, $0, 2 -> b = 2

            // --- LOOP ---
            // Lan 1:
            6'd16: q = 32'h04812000; // add $4, $4, $1 -> y = y + x
            6'd20: q = 32'h2042FFFF; // addi $2, $2, -1 -> a = a - 1
            // Lan 2:
            6'd24: q = 32'h04812000; // add $4, $4, $1 -> y = y + x
            6'd28: q = 32'h2042FFFF; // addi $2, $2, -1 -> a = a - 1

            // --- END ---
            6'd32: q = 32'h04832800; // add $5, $4, $3 -> y = y + b
            6'd36: q = 32'h08050000; // sw $5, 0($0) -> Luu ket qua vao Word 0
            6'd40: q = 32'h10060000; // lw $6, 0($0) -> Doc Word 0 ra $6 (READDATA)

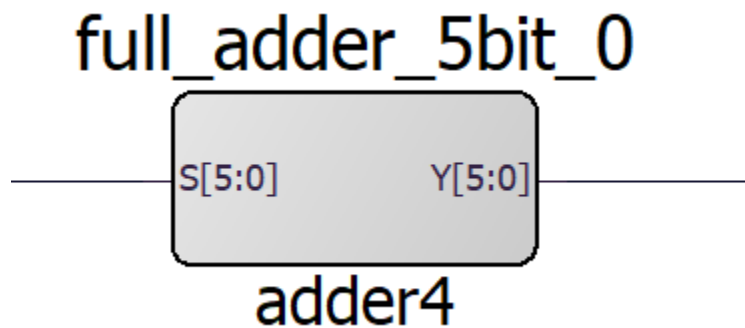
            // TRAP:
            default: q = 32'h10060000;
        endcase
    end
endmodule

```

Hình 18. Giao diện chương trình biên dịch

## VI. Verilog

### 1. Bộ cộng 6 bit:



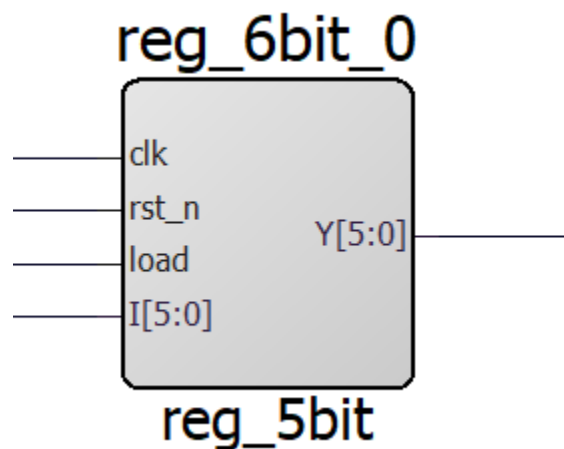
Hình 18. Bộ cộng 6 bit

```

1 module adder4(S,Y);
2     input  wire [5:0] S;
3     output wire [5:0] Y;
4     assign Y = S + 4;
5 endmodule

```

## 2. PC (PROGRAM COUNTER - BỘ ĐẾM CHƯƠNG TRÌNH):



Hình 19. Program Counter

- Program Counter (PC) của dự án này là một thanh ghi 6 bit đặc biệt có nhiệm vụ lưu trữ địa chỉ vật lý của lệnh hiện tại đang được nạp để thực thi.
- PC có chức năng điều phối quá trình nạp lệnh (Instruction Fetch).
- Địa chỉ 6 bit xuất ra từ thanh ghi PC được dẫn trực tiếp đến cổng địa chỉ của Bộ nhớ Lệnh (Instruction Memory - IMEM).
- CPU chỉ hoạt động khi tín hiệu Load được đẩy lên mức 1 và có xung clock cấp vào bộ PC.

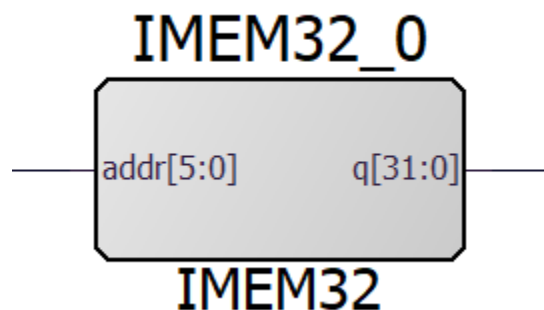


```

1 module reg_5bit (
2     input wire clk,
3     input wire rst_n,
4     input wire load,
5     input wire [5:0] I,
6     output reg [5:0] Y
7 );
8     wire [5:0] nextout;
9     assign nextout = load ? I : Y;
10 always@(posedge clk or negedge rst_n) begin
11     if(!rst_n)
12         Y <= 6'b000000;
13     else
14         Y <= nextout;
15 end
16
17 endmodule

```

### 3. ISTR\_MEM (INSTRUCTION MEMORY - BỘ NHỚ TẬP LỆNH):



Hình 20. Bộ nhớ Tập Lệnh

-IMEM là nơi lưu trữ các lệnh (dưới dạng mã Hex) mà CPU sẽ đọc và thực thi.

-Đây là bộ nhớ chỉ đọc (Read-Only Memory - ROM) được mô tả bằng mạch tổ hợp và lệnh case. Nó hoạt động như một LUT (Look-Up Table)

-Địa chỉ (addr) 6 bit có tối đa 64 vị trí bộ nhớ và chứa được tối đa 16 tập lệnh thực hiện theo tuần tự. Dữ liệu (q) 32 bit đúng với chuẩn độ dài lệnh của MIPS 32 bit.

-Về logic chương trình thì chương trình này đang thực hiện 1 bài toán tính toán cơ bản và lưu trữ kết quả:

- Khởi tạo (Địa chỉ 0 -12): Nạp giá trị ban đầu cho các thanh ghi với \$4 (y) =0; \$1 (x) =2; \$2(a) = 2; \$3(b) =2.

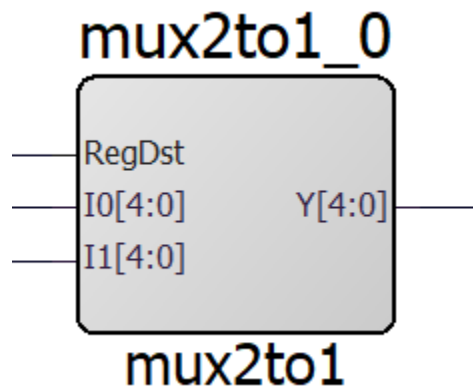
- Tính toán (Địa chỉ 16-32): Thực hiện cộng  $y=y+x$  hai lần và giảm a ( giống như mô phỏng 1 vòng lặp for nhưng được viết trả ra). Cuối cùng cộng thêm giá trị của b vào kết quả.
- Truy xuất bộ nhớ dữ liệu ( 36-40):
  - o sw: Lưu kết quả cuối cùng từ thanh ghi \$5 vào địa chỉ 0 của Data Memory.
  - o lw: Đọc lại giá trị từ địa chỉ 0 vào thanh ghi \$6. Đây là bước kiểm tra (Verify) xem việc ghi/đọc dữ liệu có thành công hay không. Việc lw ở đây là rất cần thiết vì chỉ khi lw thì readdata mới có giá trị output để kiểm tra tính đúng sai của thuật toán.

```

1 module IMEM32 (
2     input wire [5:0] addr,
3     output reg [31:0] q
4 );
5     always @(*) begin
6         case (addr[5:0])
7
8             6'd0: q = 32'h20040000; // addi $4, $0, 0 -> y = 0
9             6'd4: q = 32'h20010002; // addi $1, $0, 2 -> x = 2
10            6'd8: q = 32'h20020002; // addi $2, $0, 2 -> a = 2
11            6'd12: q = 32'h20030002; // addi $3, $0, 2 -> b = 2
12
13            6'd16: q = 32'h04812000; // add $4, $4, $1 -> y = y + x
14            6'd20: q = 32'h2042FFFF; // addi $2, $2, -1 -> a = a - 1
15
16
17            6'd24: q = 32'h04812000; // add $4, $4, $1 -> y = y + x
18            6'd28: q = 32'h2042FFFF; // addi $2, $2, -1 -> a = a - 1
19
20            6'd32: q = 32'h04832800; // add $5, $4, $3 -> y = y + b
21
22            6'd36: q = 32'h08050000; // sw $5, 0($0) -> Lưu kết quả vào Word 0
23            6'd40: q = 32'h10060000; // lw $6, 0($0) -> Đọc Word 0 ra $6 (READDATA)
24
25            default: q = 32'h00000000;
26        endcase
27    end
28 endmodule

```

#### 4. MUX 2-1 5 BIT - CƠ CHẾ CHỌN THANH GHI ĐÍCH (DESTINATION REGISTER SELECTION):



Hình 20. Mux 2 to 1 5 bit

-Quá trình xác định địa chỉ để ghi dữ liệu phức tạp hơn đáng kể do sự khác biệt giữa hai định dạng lệnh. Đối với các lệnh R-type (ví dụ: add), kết quả tính toán sẽ được ghi vào thanh ghi đích “rd”, được mã hóa tại trường Inst [15:11].<sup>17</sup> Ngược lại, trong các lệnh I-type (ví dụ: lw hay addi), dữ liệu phải được nạp vào thanh ghi “rt” nằm tại trường Inst[20:16].

-Đường dẫn dữ liệu giải quyết mâu thuẫn kiến trúc này bằng cách lắp đặt một **Bộ ghép kênh 5-bit** (MUX 2-to-1 5-bit) ngay trước cổng nhận địa chỉ ghi (Write Register - WR) của RegFile (RF).

-Bộ MUX này đóng vai trò như một ngã ba đường ray, được chuyển mạch bởi tín hiệu điều khiển RegDst (Register Destination) phát ra từ Khối Điều khiển (Control Unit -CU)

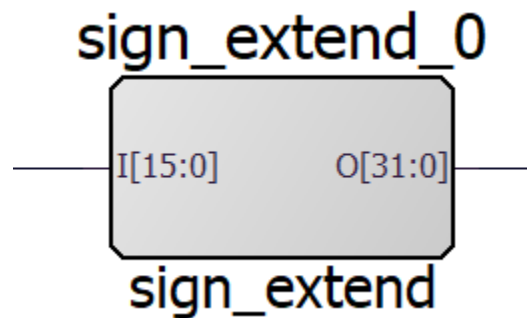
- Khi RegDst = 0, luồng tín hiệu từ Inst[20:16] được cho phép đi qua, định tuyến kết quả vào thanh ghi “rt”.
- Khi RedDst = 1, luồng tín hiệu từ Inst[15:11] được phép đi qua, định tuyến kết quả vào thanh ghi “rd”.

```

1 module mux2to1(
2     input wire [4:0] I0,
3     input wire [4:0] I1,
4     input wire RegDst,
5     output wire [4:0] Y
6 );
7
8 assign Y= RegDst ? I1 : I0;
9
10 endmodule

```

## 5. SIGN\_EXTEND - KHỎI MỞ RỘNG DẤU (SIGN EXTENSION)



Hình 21. Khối mở rộng dấu

-Kiến trúc máy tính hiện đại dựa trên nguyên tắc tính nhất quán của kích thước từ (word size) trong quá trình tính toán. ALU của bộ xử lý MIPS là một siêu mạch cộng 32-bit, yêu cầu cả hai toán hạng đưa vào ngõ vào của nó đều phải có độ rộng chính xác 32 bit.

-Tuy nhiên, như đã đề cập trong phần kiến trúc tập lệnh, trường chứa hằng số (immediate) của các lệnh I-type chỉ có kích thước 16 bit (Inst [15:0]) để nhường không gian cho Opcode và các địa chỉ thanh ghi. Sự chênh lệch kích thước này sẽ gây ra lỗi nghiêm trọng nếu không có cơ chế chuyển đổi.

-Đây là lúc Khối Mở rộng dấu (Sign Extend) phát huy tác dụng. Nhiệm vụ của khối mạch tổ hợp này là nhận chuỗi 16 bit đầu vào và sinh ra một chuỗi 32 bit đầu ra sao cho giá trị toán học của số nguyên được giữ nguyên vẹn. Trong hệ thống biểu diễn số nhị phân bù 2 (two's complement) - chuẩn mực cho việc biểu diễn số nguyên có dấu trong máy tính, việc giữ nguyên giá trị được thực hiện bằng cách sao chép bit có trọng số lớn nhất (MSB - bit thứ 15, đại diện cho bit dấu) và lấp đầy 16 bit cao nhất của không gian 32 bit.

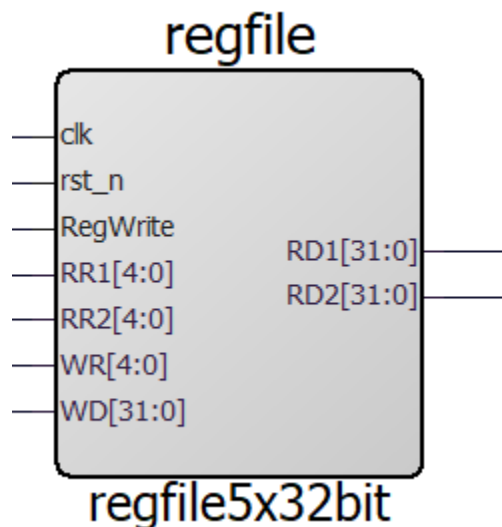
- Nếu hằng số là số dương (ví dụ 0000 0000 0000 0101 tương đương +5), bit dấu là 0. Khối Sign Extend sẽ đệm toàn bộ số 0 vào nửa trên, tạo thành 0000 0000 0000 0000 0000 0000 0101
- Nếu hằng số là số âm (ví dụ 1111 1111 1110 1001 tương đương -23 trong lệnh `addi $4, $5, -23`), bit dấu là 1. Khối này sẽ nhân bản số 1 lấp đầy phần cao, tạo thành 1111 1111 1111 1111 1111 1111 1110 1001

```

1 module sign_extend (
2     input wire [15:0] I,
3     output wire [31:0] O
4 );
5     assign O = {{16{I[15]}}, I};
6 endmodule

```

## 6. REG\_FILE - TẬP THANH GHI (REGISTER FILE - RF)



Hình 22. Register File - RF

-Sau khi được trích xuất từ IMEM, lệnh 32-bit (ký hiệu là `Inst [31:0]`) sẽ được phân tách thành các trường bit nhỏ hơn và định tuyến đến các phân hệ tương ứng. Thành phần tiếp nhận nhiều đường dẫn tín hiệu nhất là Tập thanh ghi (Register File - RF)

-Tập thanh ghi là vùng nhớ tốc độ siêu cao nằm ngay sát cạnh các khối toán học, cung cấp nguồn dữ liệu tức thời cho ALU. RF trong kiến trúc MIPS tiêu chuẩn bao gồm 32 thanh ghi, mỗi thanh ghi rộng 32 bit (hay cấu trúc 32x32). Để một bộ xử lý đơn chu kỳ hoạt động, cấu trúc phần cứng của RF phải thỏa mãn điều kiện vô cùng khắt khe: phải có khả năng đáp ứng việc đọc ra hai giá trị thanh ghi khác nhau và

ghi vào một thanh ghi thứ ba hoàn toàn độc lập, tất cả diễn ra đồng thời trong cùng một chu kỳ thời gian.

-RD1 và RD2 hoạt động độc lập, cho phép đọc cùng lúc 2 giá trị từ 2 địa chỉ khác nhau (RR1, RR2).

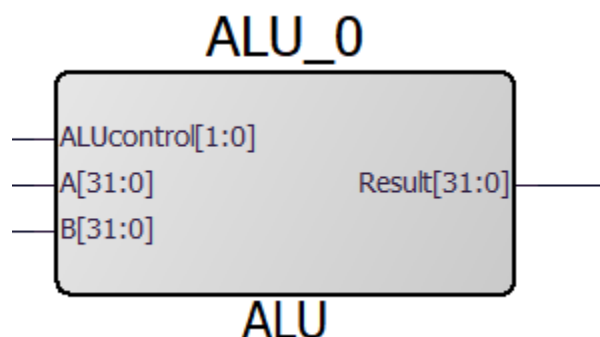
-1 Công ghi: sử dụng địa chỉ WR và dữ liệu WD, được điều khiển bởi tín hiệu cho phép ghi RegWrite.

**-Đọc (Async Read):** Việc đọc dữ liệu là **không đồng bộ** (asynchronous) vì sử dụng lệnh assign. Nghĩa là dữ liệu RD1, RD2 sẽ xuất hiện ngay lập tức khi địa chỉ RR1, RR2 thay đổi mà không cần chờ cạnh xung nhịp.

**-Ghi (Sync Write):** Việc ghi dữ liệu là **đồng bộ** (synchronous), chỉ xảy ra tại cạnh lên của clk.

```
1 module regfile5x32bit (
2     input wire clk,rst_n,RegWrite,
3     input wire [4:0] RR1,RR2,
4     input wire [4:0] WR,
5     input wire [31:0] WD,
6     output wire [31:0] RD1,RD2
7 );
8 (* syn_ramstyle = "rw_check" *) reg [31:0] reg_mem [31:0];
9 integer i;
10 always@(posedge clk or negedge rst_n) begin
11     if(!rst_n) begin
12         for (i = 0; i < 32; i = i + 1) begin
13             reg_mem[i] <= 32'b0;
14         end
15     end
16     else begin
17         if(RegWrite) begin
18             reg_mem[WR] <= WD;
19         end
20     end
21 end
22 assign RD1 = (RR1 == 0)? 32'b0 : reg_mem[RR1];
23 assign RD2 = (RR2 == 0)? 32'b0 : reg_mem[RR2];
24 endmodule
```

## 7. ALU - ARITHMETIC LOGIC UNIT (BỘ TÍNH TOÁN SỐ HỌC)



Hình 23. Bộ tính toán số học

-Trọng tâm xử lý tính toán của Datapath đặt tại Arithmetic logic unit (ALU). Trong dự án này, ALU đảm nhận trọng trách thực hiện các phép toán cộng (cho lệnh add), cộng với hằng số (cho lệnh addi), tính toán địa chỉ bộ nhớ bằng phép cộng (cho lệnh lw, sw), và phép trừ ngầm định để thực hiện so sánh (cho lệnh slt - Set on Less Than).

-Kiến trúc này sử dụng một khối cộng toàn phần 32-bit (FA32 - Full Adder 32-bit) làm lõi xử lý trung tâm. Đối với các phép cộng, đầu vào A và B được đưa thẳng vào FA32. Tuy nhiên, để linh hoạt hóa, ALU sở hữu các bộ ghép kênh nội bộ điều phối bởi các tín hiệu từ chân ALUcontrol. Ví dụ, để thực hiện phép trừ hoặc phép so sánh, cổng vào B sẽ được cho đi qua một mảng cổng NOT để đảo ngược tất cả các bit, sau đó một cờ nhớ (cin = 1) được bơm vào bộ cộng FA32 bit thấp nhất để hoàn tất nguyên tắc bù 2 ( $A-B = A + \sim B + 1$ ). Kết quả từ các phân hệ nội bộ này (Tổng, Trừ, AND, OR, So sánh) sẽ cùng chạy đến một bộ MUX lớn ở đầu ra, nơi ALUcontrol sẽ quyết định kết quả nào được phép thoát ra khỏi ALU thông qua cổng ALU result.

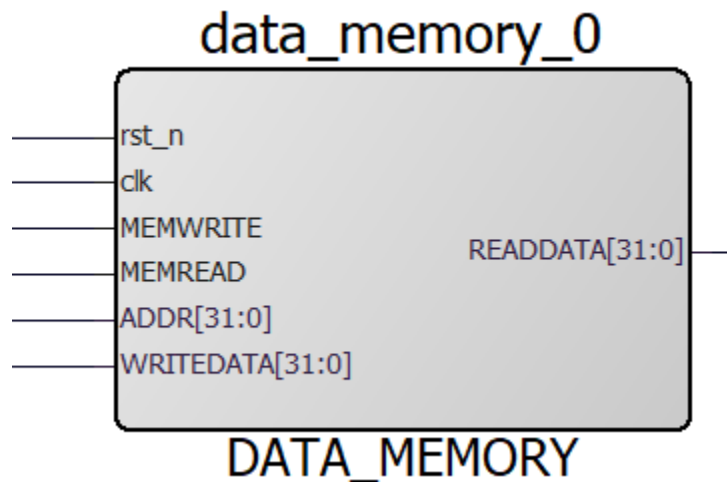
| ALUcontrol [1:0] | Tín hiệu chọn Y | Ngõ vào MUX | Phép toán thực hiện | Giải thích                  |
|------------------|-----------------|-------------|---------------------|-----------------------------|
| 2'b00            | 0               | S0          | ADD                 | Cộng: $Result=A+B$          |
| 2'b01            | 1               | I1          | SLT                 | So sánh: $Result=(A<B)?1:0$ |
| 2'b10            | 0               | S0          | ADD                 | Giống trường hợp 00         |
| 2'b11            | 1               | I1          | SLT                 | Giống trường hợp 01         |

```

1 module ALU(
2     input [31:0] A,B,
3     input [1:0] ALUCONTROL,
4     output [31:0] RESULT
5 );
6     wire [31:0] sum;
7     wire [31:0] sub;
8     wire [31:0] slt;
9
10    assign sum = A+B;
11    assign sub = A-B;
12    assign slt = {31'b0, sub[31]};
13
14    assign RESULT = (ALUCONTROL[0] == 0) ? sum : slt;
15 endmodule

```

## 8. DATA MEMORY - BỘ NHỚ DỮ LIỆU (DMEM)



Hình 24. Data Memory

-Bộ nhớ Dữ liệu (Data Memory - DMEM) là phân hệ lưu trữ dung lượng lớn, đóng vai trò chứa dữ liệu mảng, các biến toàn cục, và cấu trúc ngăn xếp (stack) của phần mềm. Tuân thủ chặt chẽ mô hình Load/Store của MIPS, DMEM chỉ có thể được tương tác thông qua hai lệnh chuyên dụng: lw (Load Word - Đọc dữ liệu ra) và sw (Store Word - Ghi dữ liệu vào). Bất kỳ lệnh nào khác như tính toán số học hay logic đều không được phép chạm đến bộ phận này.

-Giao diện kết nối của DMEM vào Datapath bao gồm ba cổng chính <sup>4</sup>:



1. **Cổng Địa chỉ (Address):** Để biết cần đọc hoặc ghi tại vị trí nào trong không gian bộ nhớ khổng lồ, DMEM nhận tín hiệu trực tiếp từ đầu ra ALU result của ALU. Trong các lệnh bộ nhớ, ALU đóng vai trò là mạch tính toán địa chỉ chuyên dụng bằng cách lấy địa chỉ cơ sở (base address) từ thanh ghi cộng với hằng số độ dời (offset).
2. **Cổng Ghi Dữ liệu (Write Data):** Dữ liệu cần được lưu trữ (trong trường hợp lệnh sw) được lưu ở thanh ghi rt. Tín hiệu chứa dữ liệu này được xuất ra từ cổng Read Data 2 (RD2) của Register File và được đầu nối trực tiếp, băng qua toàn bộ Datapath, tiến thẳng đến cổng Write Data của DMEM.
3. **Cổng Đọc Dữ liệu (Read Data):** Cổng này liên tục xuất ra nội dung của ô nhớ tương ứng với địa chỉ đang tồn tại ở cổng Address, sẵn sàng cung cấp dữ liệu cho quá trình ghi ngược.

-Tuy nhiên, do DMEM chứa dữ liệu quan trọng của chương trình, nó không thể để các dữ liệu ngẫu nhiên rò rỉ hoặc vô tình bị ghi đè trong các chu kỳ lệnh không liên quan (ví dụ lệnh add tính toán ra một ALU result không phải là một địa chỉ hợp lệ). Nhằm bảo vệ tính toàn vẹn của trạng thái, hoạt động của DMEM bị khóa chặt bằng hai tín hiệu điều khiển độc lập: MemRead và MemWrite.

-MEMWRITE: Cho phép ghi dữ liệu (WRITEDATA) vào bộ nhớ tại địa chỉ ADDR vào cạnh lên của xung nhịp.

-MEMREAD: Cho phép đọc dữ liệu từ bộ nhớ.

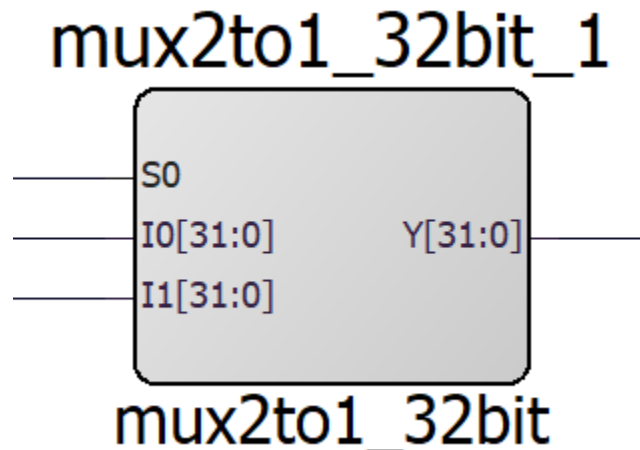
- Việc truy xuất dữ liệu từ DMEM là thao tác không đồng bộ (asynchronous read). Chỉ khi tín hiệu MemRead = 1 được phát ra từ Control Unit, dữ liệu mới được đẩy ra khỏi chip nhớ. Ở các trạng thái khác, cổng ra thường bị đưa về trạng thái trở kháng cao (High Impedance - Z) để tránh tranh chấp tín hiệu trên bus dữ liệu. Nhưng do ở đây để Synthesis và thực thi trên kit được thì bọn em đã để cổng ra mặc định ở trạng thái 32'd0.
- Việc ghi dữ liệu là thao tác đồng bộ hoàn toàn (synchronous write). Dữ liệu chỉ được lưu vào bộ nhớ tại cạnh lên của clk khi có tín hiệu MEMWRITE. Điều này đảm bảo dữ liệu không bị ghi đè ngẫu nhiên do nhiễu tín hiệu.
- Chỉ thị syn\_ramstyle: Giúp công cụ tổng hợp FPGA tối ưu hóa việc quản lý xung đột đọc/ghi (Read-during-write), đảm bảo tính toàn vẹn của dữ liệu.

```

1 module DATA_MEMORY(
2     input rst_n, clk,
3     input [31:0] ADDR,
4     input [31:0] WRITEDATA,
5     input MEMWRITE,
6     input MEMREAD,
7     output [31:0] READDATA
8 );
9 (* syn_ramstyle = "rw_check" *) reg [31:0] WORD [31:0];
10 integer i;
11 always@(posedge clk or negedge rst_n) begin
12     if(!rst_n) begin
13         for(i = 0; i < 32; i = i + 1) begin
14             WORD[i] <= i;
15         end
16     end
17     else if (MEMWRITE)
18         WORD[ADDR] <= WRITEDATA;
19 end
20
21 assign READDATA = (MEMREAD) ? WORD[ADDR] : 32'b0;
22 endmodule

```

## 9. MUX 2-1 32 BIT - BỘ CHỌN NGUỒN DỮ LIỆU GHI (WRITE-BACK MUX)



Hình 25. Mux 2-1 32 bit

-Vòng đời của đa số các lệnh đều kết thúc bằng việc cập nhật trạng thái kết quả quay trở lại hệ thống thanh ghi. Quá trình này, gọi là Ghi ngược (Write Back), là

bước hiện thực hóa kết quả tính toán để sử dụng cho các lệnh kế tiếp trong tương lai.

-Sự hiện diện của các khối chức năng khác nhau trong Datapath tạo ra nhiều nguồn dữ liệu tiềm năng cần được lưu trữ:

- Đối với các lệnh số học và logic (add, addi, slt), dữ liệu cần lưu lại là thành quả tính toán của ALU, đang nằm sẵn trên đường truyền ALU result.
- Đối với lệnh lấy dữ liệu từ bộ nhớ (lw), kết quả của ALU chỉ đóng vai trò là địa chỉ trung gian. Dữ liệu thực sự cần thiết đang được đẩy ra từ cổng Read Data của Data Memory.

-Hai luồng dữ liệu 32-bit này va chạm với nhau tại cửa ngõ của Tập thanh ghi, bởi vì cổng Write Data của Register File chỉ có thể tiếp nhận một tín hiệu duy nhất.

-Giải pháp cơ học trong không gian kiến trúc là việc lắp đặt một Bộ ghép kênh 32-bit cuối cùng, được điều phối bởi tín hiệu MemToReg (Memory to Register).

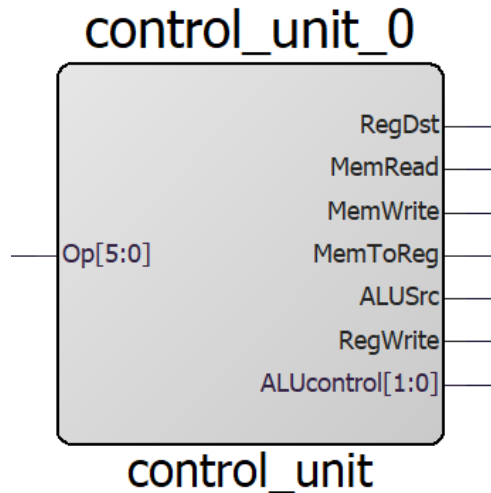
- Khi thực thi các lệnh toán học, Khối Điều khiển phát tín hiệu MemToReg = 0. Bộ MUX sẽ ưu tiên cho đường dẫn từ ALU result bẻ ngược vòng về cổng Write Data của RF.
- Khi lệnh đang chạy là lw, MemToReg = 1 được kích hoạt. Bộ MUX gạt luồng tín hiệu từ ALU sang một bên, thiết lập cầu nối liên thông từ ngõ ra của Data Memory về hệ thống thanh ghi trung tâm RF.

-Cuối cùng, khi dữ liệu chuẩn đã hội tụ tại cửa, kết hợp với địa chỉ thanh ghi mục tiêu đã được quyết định bởi MUX RegDst ở trên, và dưới sự cho phép bằng tín hiệu RegWrite = 1, mọi thứ đã sẵn sàng. Ngay khoảnh khắc sườn lên của xung nhịp clock quét qua, dữ liệu mới sẽ được chốt cứng vào thanh ghi, hoàn tất quá trình của một chu kỳ lệnh đơn chu kì.

- Cổng I0: Nối với đầu ra Result của ALU (Dành cho các lệnh tính toán như add, addi, sub).
- Cổng I1: Nối với đầu ra READDATA của Data Memory (RAM) (Dành cho lệnh nạp dữ liệu lw).
- Tín hiệu chọn S0: Nối với tín hiệu điều khiển MemToReg từ control\_unit.
- Khi S0 (MemToReg) = 1: MUX chọn I1. CPU sẽ lấy dữ liệu vừa đọc từ RAM để ghi vào thanh ghi (Đây là hoạt động của lệnh Load Word - lw).

- Khi  $S0 (MemToReg) = 0$ : MUX chọn I0. CPU sẽ lấy kết quả tính toán trực tiếp từ ALU để ghi vào thanh ghi (Đây là hoạt động của các lệnh tính toán như add, addi, slt).

## 10. CONTROL UNIT - BỘ ĐIỀU KHIỂN (CU)



Hình 26. Control Unit

-Khối điều khiển nhận đầu vào là đoạn mã lệnh Opcode (6 bit cao nhất Inst [31:26]) từ tín hiệu lệnh vừa được nạp và phân tích tức thời. Phản ứng với chuỗi bit đó, CU phát ra chuỗi tín hiệu điều khiển bao gồm hàng loạt các tín hiệu điện tĩnh (static control signals) để chuyển mạch hàng loạt bộ MUX, kích hoạt các cổng đọc/ghi, và thiết lập trạng thái ALU trên toàn bộ Datapath ngay tại đầu mỗi chu kỳ, đảm bảo dòng dữ liệu chảy đúng lộ trình.

- `RegWrite` và `RegDst`: Quyết định việc ghi kết quả vào tập thanh ghi.
- `MemRead`, `MemWrite`, `MemToReg`: Nhóm tín hiệu tương tác với RAM. Lưu ý `MemToReg` và `MemRead` cùng chung logic `Op[2]`, điều này khớp với lệnh `lw` (Load Word) – vừa đọc RAM vừa chuẩn bị ghi vào Thanh ghi.
- `ALUcontrol`: `ALUcontrol[1]` bị chốt cứng ở mức 1. ALU sẽ chỉ chạy hai chế độ: `2'b10` (ADD) hoặc `2'b11` (SLT).

```

1 module control_unit (
2     input  [5:0] Op,
3     output      RegDst,
4     output      MemRead,
5     output      MemWrite,
6     output      MemToReg,
7     output      ALUSrc,
8     output      RegWrite,
9     output [1:0] ALUcontrol
10 );
11
12     assign RegDst = Op[0] | Op[4];
13     assign MemRead = 1'b1 & Op[2];
14     assign MemWrite = 1'b1 & Op[1];
15     assign MemToReg = 1'b1 & Op[2];
16     assign ALUSrc = Op[1] | Op[2] | Op[3];
17     assign RegWrite = Op[0] | Op[2] | Op[3] | Op[4];
18     assign ALUcontrol[0] = Op[4] ? 1'b1 : 1'b0;
19     assign ALUcontrol[1] = 1'b1;
20
21 endmodule

```

## VII. Testbench



```

6'd0: q = 32'h20040000; // addi $4, $0, 0 -> y = 0
6'd4: q = 32'h20010002; // addi $1, $0, 2 -> x = 2
6'd8: q = 32'h20020002; // addi $2, $0, 2 -> a = 2
6'd12: q = 32'h20030002; // addi $3, $0, 2 -> b = 2

6'd16: q = 32'h04812000; // add $4, $4, $1 -> y = y + x
6'd20: q = 32'h2042FFFF; // addi $2, $2, -1 -> a = a - 1

6'd24: q = 32'h04812000; // add $4, $4, $1 -> y = y + x
6'd28: q = 32'h2042FFFF; // addi $2, $2, -1 -> a = a - 1

6'd32: q = 32'h04832800; // add $5, $4, $3 -> y = y + b

6'd36: q = 32'h08050000; // sw $5, 0($0) -> Lưu kết quả vào Word 0
6'd40: q = 32'h10060000; // lw $6, 0($0) -> Đọc Word 0 ra $6 (READDATA)

```

- Từ waveform và file IMEM đã được nạp sẵn instruction từ trước, ta có thể thấy được rằng chương trình sẽ in ra màn hình kết quả sau 10 lần nhấn tay, phù hợp với 11 lệnh đã được tạo ( vì sau khi nhấn reset thì instruction thứ nhất đã được thực thi và đây là cấu trúc CPU MIPS đơn chu kỳ ).

```

1  `timescale 1ns / 1ps
2
3  module tb_cpu();
4      reg btn_debug;
5      reg clk;
6      reg clk_50m;
7      reg rst;
8
9      wire [5:0] Op;
10     wire tx_pin;
11
12     cpu uut (
13         .btn_debug (btn_debug),
14         .clk        (clk),
15         .clk_50m    (clk_50m),
16         .rst        (rst),
17         .Op         (Op),
18         .tx_pin     (tx_pin)
19     );
20     initial begin
21         clk_50m = 1'b0;
22         forever #10 clk_50m = ~clk_50m;
23     end
24     initial begin
25         rst      = 1'b0;
26         clk      = 1'b0;

```

```

27 btn_debug = 1'b0;
28 #100;
29 rst = 1'b1;
30 #100;
31 repeat (11) begin
32     clk = 1'b1;
33     #25_000_000;
34     clk = 1'b0;
35     #5_000_000;
36 end
37 btn_debug = 1'b1;
38 #25_000_000;
39 btn_debug = 1'b0;
40 #10_000_000;
41 $stop;
42 end
43 endmodule

```

## VIII. Constraint

### 1. I/O Constraint

|    | Port Name | Direction | I/O Standard | Pin Number | Locked                              | Macro Cell | Ba |
|----|-----------|-----------|--------------|------------|-------------------------------------|------------|----|
| 1  | btn_debug | INPUT     | LVC MOS18    | U18        | <input checked="" type="checkbox"/> | INBUF      |    |
| 2  | clk       | INPUT     | LVC MOS18    | T19        | <input checked="" type="checkbox"/> | INBUF      |    |
| 3  | clk_50m   | INPUT     | LVC MOS18    | R18        | <input checked="" type="checkbox"/> | INBUF      |    |
| 4  | ▼ Op      |           |              |            | <input type="checkbox"/>            |            |    |
| 5  | Op[0]     | OUTPUT    | LVC MOS18    | T18        | <input checked="" type="checkbox"/> | OUTBUF     |    |
| 6  | Op[1]     | OUTPUT    | LVC MOS18    | V17        | <input checked="" type="checkbox"/> | OUTBUF     |    |
| 7  | Op[2]     | OUTPUT    | LVC MOS18    | U20        | <input checked="" type="checkbox"/> | OUTBUF     |    |
| 8  | Op[3]     | OUTPUT    | LVC MOS18    | U21        | <input checked="" type="checkbox"/> | OUTBUF     |    |
| 9  | Op[4]     | OUTPUT    | LVC MOS18    | AA18       | <input checked="" type="checkbox"/> | OUTBUF     |    |
| 10 | Op[5]     | OUTPUT    | LVC MOS18    | V16        | <input checked="" type="checkbox"/> | OUTBUF     |    |
| 11 | rst       | INPUT     | LVC MOS18    | U17        | <input checked="" type="checkbox"/> | INBUF      |    |
| 12 | tx_pin    | OUTPUT    | LVC MOS18    | E15        | <input checked="" type="checkbox"/> | OUTBUF     |    |

### 2. Timing Constraint

```
create_clock -name {clk-50m} -period 20 -waveform {0 10} [ get_ports { clk_50m } ]
```

Yêu cầu thiết kế: Mạch hoạt động tối thiểu ở tần số 50MHz.

## IX. Báo cáo tài nguyên

```
Design: E:\verilog_study\cpu32bit\designer\cpu\cpu
Finished: Mon Apr 20 18:26:10 2026
Total CPU Time:      00:00:33      Total Elapsed Time: 00:00:36
Total Memory Usage: 1755.4 Mbytes
```

o - o - o - o - o - o

### Resource Usage

| Type                     | Used | Total | Percentage |
|--------------------------|------|-------|------------|
| 4LUT                     | 1168 | 93516 | 1.25       |
| DFF                      | 1220 | 93516 | 1.30       |
| Logic Element            | 1822 | 93516 | 1.95       |
| I/Os using I/O Registers | 0    | 80    | 0.00       |
| I/O Register Flip-Flops  | 0    | 240   | 0.00       |
| -- Input I/O Flip-Flops  | 0    | 80    | 0.00       |
| -- Output I/O Flip-Flops | 0    | 80    | 0.00       |
| -- Enable I/O Flip-Flops | 0    | 80    | 0.00       |

### I/O Placement

| Type   | Count | Percentage |
|--------|-------|------------|
| Locked | 11    | 100.00%    |
| Placed | 0     | 0.00%      |

## X. Timing Analysis

### Clock Details Summary

| Clock Domain | Period (ns) | Frequency (MHz) | Required Period (ns) | Required Frequency (MHz) | External Setup (ns) | External Hold (ns) | Min Clock-To-Out (ns) | Max Clock-To-Out (ns) |
|--------------|-------------|-----------------|----------------------|--------------------------|---------------------|--------------------|-----------------------|-----------------------|
| clk-50m      | 2.617       | 382.117         | 20.000               | 50.000                   | 0.965               | 0.681              | 7.324                 | 12.987                |

Nhận xét:

- Dải nhiệt độ hoạt động: 0 - 100 độ C
  - Thời gian trễ thực tế (Actual Period): 2,617 ns
  - Thời gian hoàn thành theo yêu cầu: 20 ns
- => Slack = 17,383 ns => Design chạy tốt
- Tần số tối đa đạt được là 382MHz.

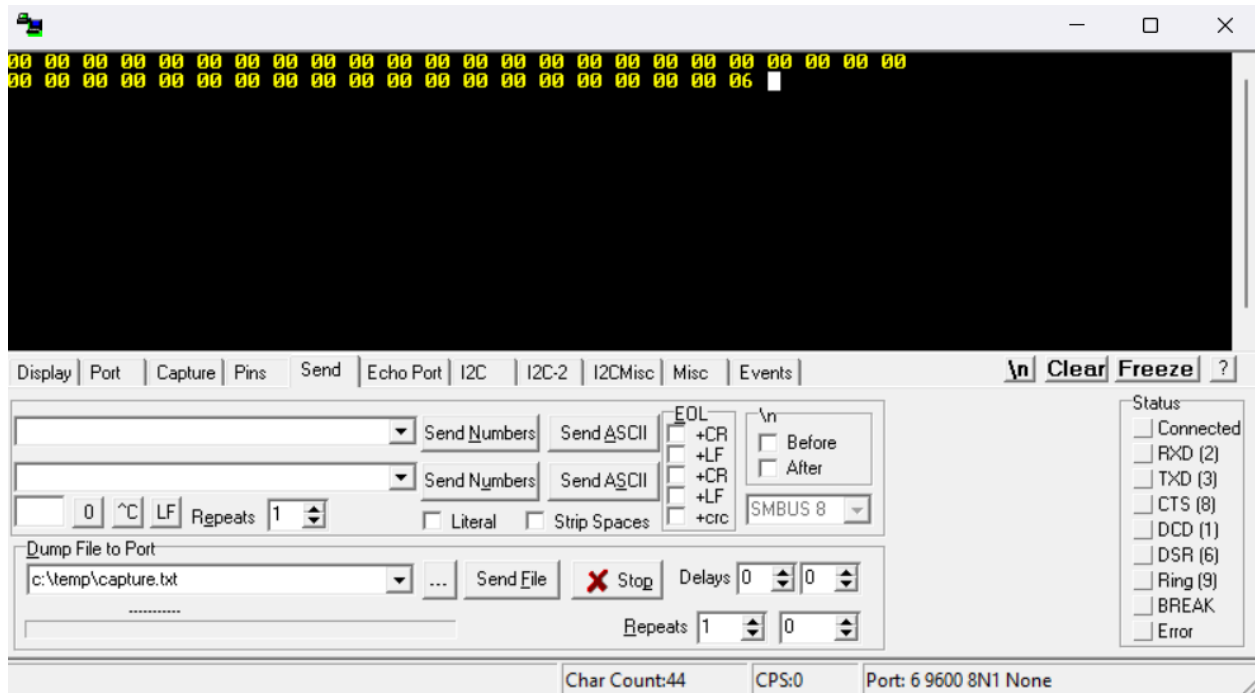


|    | Source Pin           | Sink Pin                      | Delay (ns) | Slack (ns) | Arrival (ns) | Required (ns) | Setup (ns) | Minimum Period (ns) | Skew (ns) |
|----|----------------------|-------------------------------|------------|------------|--------------|---------------|------------|---------------------|-----------|
| 1  | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[14][18]:EN | 8.940      |            | 14.323       |               | 0.128      | 9.189               | 0.121     |
| 2  | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][22]:EN | 8.956      |            | 14.339       |               | 0.128      | 9.158               | 0.074     |
| 3  | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][0]:EN  | 8.970      |            | 14.353       |               | 0.128      | 9.132               | 0.034     |
| 4  | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][12]:EN | 8.970      |            | 14.353       |               | 0.128      | 9.132               | 0.034     |
| 5  | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][24]:EN | 8.969      |            | 14.352       |               | 0.128      | 9.131               | 0.034     |
| 6  | reg_32bit_0/Y[2]:CLK | data_memory_0/WORD[14][18]:EN | 8.844      |            | 14.227       |               | 0.128      | 9.093               | 0.121     |
| 7  | reg_32bit_0/Y[2]:CLK | data_memory_0/WORD[12][22]:EN | 8.860      |            | 14.243       |               | 0.128      | 9.062               | 0.074     |
| 8  | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[20][12]:EN | 8.874      |            | 14.257       |               | 0.136      | 9.049               | 0.039     |
| 9  | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[20][24]:EN | 8.874      |            | 14.257       |               | 0.136      | 9.049               | 0.039     |
| 10 | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[0][28]:EN  | 8.857      |            | 14.240       |               | 0.136      | 9.039               | 0.046     |
| 11 | reg_32bit_0/Y[2]:CLK | data_memory_0/WORD[12][0]:EN  | 8.874      |            | 14.257       |               | 0.128      | 9.036               | 0.034     |
| 12 | reg_32bit_0/Y[2]:CLK | data_memory_0/WORD[12][12]:EN | 8.874      |            | 14.257       |               | 0.128      | 9.036               | 0.034     |
| 13 | reg_32bit_0/Y[2]:CLK | data_memory_0/WORD[12][24]:EN | 8.873      |            | 14.256       |               | 0.128      | 9.035               | 0.034     |
| 14 | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][29]:EN | 8.781      |            | 14.164       |               | 0.136      | 9.022               | 0.105     |
| 15 | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][8]:EN  | 8.780      |            | 14.163       |               | 0.136      | 9.021               | 0.105     |
| 16 | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][5]:EN  | 8.826      |            | 14.209       |               | 0.136      | 9.014               | 0.052     |
| 17 | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][7]:EN  | 8.826      |            | 14.209       |               | 0.136      | 9.014               | 0.052     |
| 18 | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][9]:EN  | 8.826      |            | 14.209       |               | 0.136      | 9.014               | 0.052     |
| 19 | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][14]:EN | 8.812      |            | 14.195       |               | 0.128      | 8.991               | 0.051     |
| 20 | reg_32bit_0/Y[3]:CLK | data_memory_0/WORD[12][31]:EN | 8.812      |            | 14.195       |               | 0.128      | 8.991               | 0.051     |

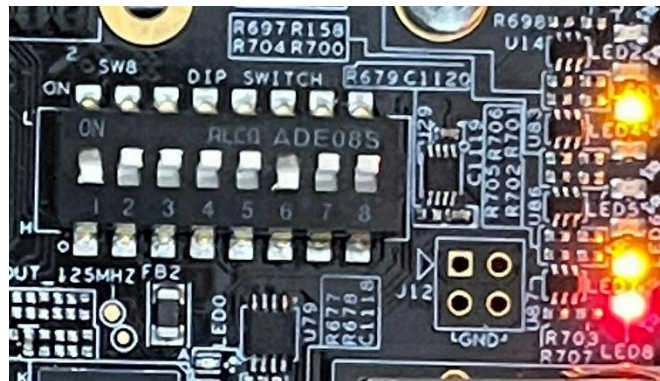
Nhận xét: Slack dương => không tồn tại violation

## XI. Kết quả:

### 1. Trên màn hình Realterm



### 2. Trên FPGA



Đèn Led đọc từ dưới lên trên, đang thể hiện trạng thái 000100, đại diện cho lệnh LW

=> như vậy, dự án hoạt động đúng như những gì đã thiết kế ban đầu và đúng theo mô phỏng.

# Kết luận

## Tổng kết và Bài học kinh nghiệm:

Thông qua quá trình hiện thực hóa bộ vi xử lý MIPS 32-bit đơn chu kỳ và tích hợp module giao tiếp ngoại vi, nhóm chúng em đã đạt được những bước tiến quan trọng về tư duy kỹ thuật:

- **Kỹ năng thiết kế phần cứng:** Làm chủ phương pháp hiện thực hóa các khối kiến trúc cốt lõi của máy tính (ALU, Register File, Memory, Control Unit) bằng ngôn ngữ Verilog, chuyển từ việc mô phỏng các module đơn lẻ sang tích hợp thành một hệ thống Datapath hoàn chỉnh.
- **Hiểu biết sâu sắc về Kiến trúc máy tính:** Nắm vững luồng đi của dữ liệu và cơ chế điều khiển bằng phần cứng, hiểu rõ cách một mã lệnh (Opcode) được bộ điều khiển giải mã và thực thi thông qua việc luân chuyển dữ liệu trên thực tế.
- **Tư duy giải quyết vấn đề thực tế:** Nhận thức rõ khoảng cách giữa lý thuyết lý tưởng và môi trường thiết kế thực tế. Đặc biệt là kỹ năng xử lý nhiễu nút nhấn (Debounce) và thiết kế bộ chia tần số (Baud Rate Generator) để hệ thống có thể giao tiếp ổn định qua giao thức UART.
- **Trực quan hóa tư duy logic:** Chuyển đổi thành công các mô hình lý thuyết phức tạp như Máy trạng thái hữu hạn (FSM) dùng trong truyền tải dữ liệu, hay Bảng chân trị (Truth Table) dùng trong giải mã tập lệnh thành các luồng điều khiển hệ thống thực tế một cách logic và khoa học.

## Tài liệu tham khảo

[1] H. V. Thang, "Hardware Description Language – Lecture Slides," Faculty of Electronics & Telecommunication Engineering, Danang University of Science and Technology, Jan. 2026.

[2] "Verilog Tutorial," VLSI Verify .[Online]. Available: <https://vlsiverify.com/verilog/> [Accessed: Apr. 21, 2026].

